



# A customized residual neural network and bi-directional gated recurrent unit-based automatic speech recognition model

Selim Reza <sup>a</sup> , Marta Campos Ferreira <sup>a</sup> , J.J.M. Machado <sup>b</sup> ,  
João Manuel R.S. Tavares <sup>b</sup>  

Show more 



Outline



Share



Cite

<https://doi.org/10.1016/j.eswa.2022.119293> 

[Get rights and content](#) 

Under a Creative Commons [license](#) 

Open access

## Highlights

- ResCNN before Bi-GRU cells improves the accuracy and reduces the training time.
- Layer normalization is more effective than batch normalization.
- The larger the training dataset, the better the model's performance.
- For optimizer, AdamW, instead of traditional Adam, leads to better results.
- GELU, instead of ReLU, works well as the activation function.

## Abstract

Speech recognition aims to convert human speech into text and has applications in security, healthcare, commerce, automobiles, and technology, just to name a few. Inserting residual neural networks before recurrent neural network cells improves accuracy and cuts training time by a good margin. Furthermore, layer normalization instead of batch normalization is more effective in model training and performance enhancement. Also, the size of the datasets presents tremendous influences in achieving the best performance. Leveraging these tricks, this article proposes an automatic speech recognition model with a stacked five layers of customized Residual Convolution Neural Network and seven layers of Bi-Directional Gated Recurrent Units, including a logarithmic *softmax* for the model output. Each of them incorporates a learnable per-element affine parameter-based layer normalization technique. The training and testing of the new model were conducted on the LibriSpeech corpus and LJ Speech dataset. The experimental results demonstrate a character error rate (CER) of 4.7 and 3.61% on the two datasets, respectively, with only 33 million parameters without the requirement of any external language model.



## Keywords

Deep learning; Layer normalization; Character error rate; Word error rate; LibriSpeech corpus; LJ speech dataset

---

## 1. Introduction

Voice technology is currently employed in many industries, allowing businesses and consumers to facilitate digitization and automation. For example, speech recognition improves the safety and efficiency of vehicles by enabling voice-activated navigation systems and enhancing search capabilities. Besides, voice commands to access virtual agents, particularly on mobile devices, help the call centres transcribe a colossal

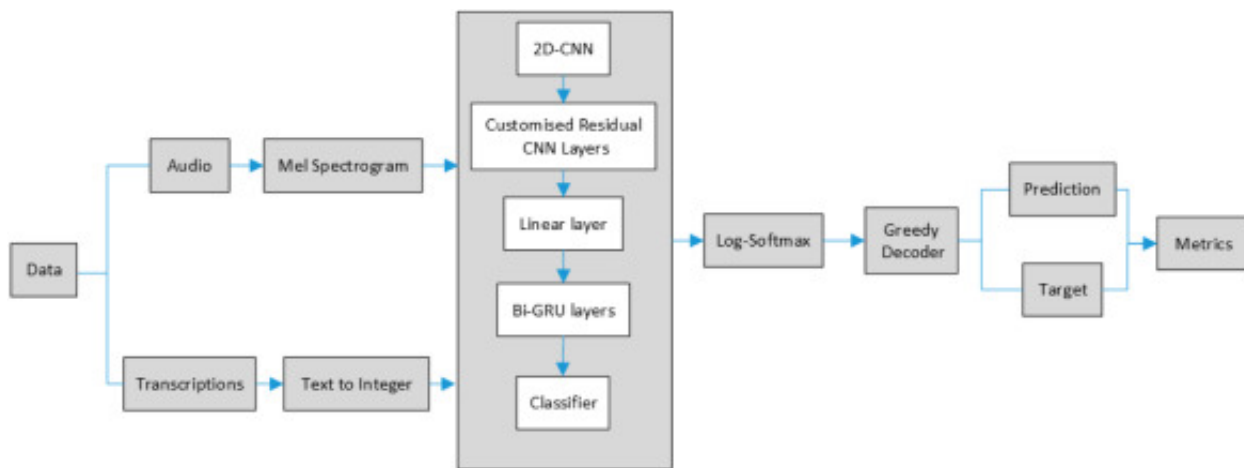
number of phone calls and identify call patterns, queries and issues. Additionally, voice-based authentication generally provides an additional security level ([Junqua & Haton, 2012](#)).

However, speech recognition is one of the most challenging computer science topics due to the difficulties of separating similar phonetically sentences and smearing problems. Various algorithms and computational approaches have been proposed to convert speech to text and increase transcription accuracy ([Yu & Deng, 2016](#)). Neural networks, particularly Deep Learning (DL) algorithms, are best suited for these purposes. They can efficiently process vast amounts of data, identify hidden patterns between features, learn the mapping function, and then adjust it based on the loss function using the gradient descent technique. While neural networks are more accurate and can handle more inputs, they achieve lower performance efficiency than classic language models since they require longer training time and more computational resources ([Benzeghiba et al., 2007](#)).

Nowadays, transformer-based models are state-of-the-art as they have demonstrated superior performances compared to other baselines. However, these models are computationally expensive. Therefore, multiple (8 – 16) graphical processing units (GPUs) in parallel with synchronous stochastic gradient (SGD) are generally applied to train this kind of model ([Likhomanenko et al., 2021](#); [Park et al., 2019](#)). Residual Convolution Neural Networks (ResNets) reduce training time while improving the model's performance. Moreover, batch normalization is simply an additional network layer between two hidden layers. Its function is to normalize the outputs of the first hidden layer before passing them onto the next hidden layer as input, helping keep the network stable throughout training. Unlike batch normalization, layer normalization calculates the normalization statistics directly from the summed inputs to the neurons within a hidden layer, ensuring that no additional dependencies between training examples are introduced ([Santurkar, Tsipras, Ilyas, & Madry, 2018](#)).

[Karita et al. \(2019\)](#), [Moritz, Hori, and Le Roux \(2019\)](#) and [Wang, Guan, and Li \(2018\)](#) proposed different architectures for solving automatic speech recognition (ASR) problems on LibriSpeech datasets. [Wang et al. \(2018\)](#) integrated two-dimensional invariant convolution layers and groups of residual convolution blocks and Bi-GRUs alongside the Connectionist Temporal Classification (CTC) loss. The authors used a subset of the LibriSpeech datasets for conducting the experiments. [Karita et al. \(2019\)](#) introduced a speech auto-encoder loss function and proposed an encoder–decoder architecture for speech recognition tasks, performing the model training and

validation on a subset of the LibriSpeech datasets. [Moritz et al. \(2019\)](#) proposed a triggered attention mechanism with a CTC-based classifier to control the attention mechanism's activation. The proposed algorithm differs from the previously mentioned models in terms of architecture. [Fig. 1](#) illustrates the overall concept of the proposed model. It relied on customized ResNets without any invariant convolution layers; particularly, it contains five layers of customized ResNets with seven layers of Bi-GRUs. For training and testing the new model, the LibriSpeech ASR corpus, which contains **1000** hours of audio speech, and the LJ Speech dataset, which includes approximately **24** hours of audio clips of a single speaker, were used. The experimental results demonstrated improvements in reducing the computational cost while providing a minimum CER of only 4.7 and 3.61%, respectively, on the used datasets without the need for any external language model. A CER of 4.7% means that 95.3% characters, which include not only letters but also punctuation and spaces, were correctly recognized. The main contributions of this research are as follows:



[Download:](#) [Download high-res image \(128KB\)](#)

[Download:](#) [Download full-size image](#)

Fig. 1. Illustration of the overall concept of the proposed model.

- It proposes a stacked five layers of customized ResNets and seven layers of Bi-GRUs, each including a layer normalization based on a learnable element-wise affine parameters approach without the requirement of external language models.
- The inclusion of the Gaussian error linear unit (GELU) layer and the dense and dropout layers for the classification tasks showed its worthiness in performance enhancement.
- It demonstrates that the volume of the training data significantly affects

the model's output.

- It also demonstrates that the proposed architecture of ResNets helps the model to achieve performance improvement with five or more layers of Bi-GRUs, which is the opposite of the findings presented in [Wang et al. \(2018\)](#).

The remaining sections of this article are the following: Section 2 presents selected state-of-the-art works along with their performances and limitations; Section 3 introduces the proposed ResNets and the Bi-GRUs based model; the experimental setups and results are presented in Section 4; Section 6 is devoted to discuss the overall performance and feasibility of proposed model in automatic speech recognition tasks; and finally, Section 7 outlines the conclusions.

## 2. Related works

Three types of ASR models are common in the literature: (i) CTC-based, (ii) transformer-based, and (ii) recurrent neural networks (RNNs)-based models. [Lee and Watanabe \(2021\)](#) presented a conformer model with an intermediate CTC loss connected to an encoder network's intermediate layer showing that training regularization and performance improvements reached a Word error rate (WER) and CER of 9.9 and 5.2% on the WSJ and AISHELL-1 corpus, respectively. The comparisons showed that state-of-the-art performance is achievable with only 12 layers of conformer (118 million parameters) compared with 48 layers of a transformer-based model. Although the authors reduced the computational expenses to a good margin, more improvements are still required.

[Li, Xu, and Zhang \(2021\)](#) proposed a linear attention-based conformer that reduced the number of parameters of the conformer-based model by 50%, guaranteeing faster training, but still achieving a CER of 4.88% on the same datasets. Two factors were responsible for reducing the number of parameters: the application of (i) multi-head linear self-attention and (ii) low-rank matrix factorization techniques. [Majumdar et al. \(2021\)](#) proposed the Citrinet model, which consists of a deep residual neural model using one-dimensional time-channel separable convolutions. The authors combined it with sub-word encoding, squeeze-and-excitation and convolutional CTC. The results demonstrated very high accuracy with only 37.2 million parameters on LibriSpeech, MLS, TED-LIUM 2, and AISHELL datasets without any requirement of external language models.

Zhao et al. (2021) proposed a semantic correction model using pre-trained BERT (Liu et al., 2019) with six encoder and decoder layers. It effectively reduced the CER by 21.7% compared with 3gram, 4gram, and RNN-based models. Fan, Chu, Chang, Xiao, and Alwan (2021), for both the encoder and decoder architectures, proposed convolution-augmented self-attention blocks. Expanding the acoustic boundary for each token demonstrated an increase in CTC alignment's robustness, and using an iterative loss function enhanced the gradient update for low-layer parameters. These concepts led to a 7 to 21% improvement compared to the other baselines by achieving WER of 3.1 and 7.2% on Librispeech *Test\_Clean* and *Test\_Other* datasets, respectively, and a CER of 5.4% on the AISHELL-1 test dataset.

Karita et al. (2019) introduced semi-supervised speech and text autoencoders to improve the performance further. The work presented a loss function called maximum mean discrepancy, which achieved a CER and a WER of 10.4 to 8.4% and 20.6 to 18.0% on the LibriSpeech *Test\_Clean* dataset, respectively. Baidu's Deep Speech (Hannun et al., 2014) and Google's Listen Attend Spell (Chan, Jaitly, Le, & Vinyals, 2016) are two of the most famous pioneering speech recognition models on RNNs. However, the authors employed distinct techniques to model speech recognition problems. The first one predicted speech transcriptions using the CTC loss function, which achieved a WER of 13.25% on the Librispeech datasets. The second model for prediction employed a sequence-to-sequence network architecture, which demonstrated a WER of only 6.5% on the same datasets.

Han et al., 2020 and Krivan et al., 2020 incorporated depthwise separable convolution layers to further reduce the computational costs while maintaining adequate accuracy on the Librispeech datasets. The standard convolution requires around 9 times more multiplications than the depthwise separable convolution. Hence, the authors achieved tremendous accuracy with much fewer parameters; for example, a WER of 7 % on Librispeech *Test\_Other* with only 10.8M parameters. However, for small networks, these models might end up with too few parameters and fail to learn appropriately during training. Also, the depthwise separable convolution technique solely relies on cross-kernel correlations and a more efficient separation of regular convolutions is required to address the demands (Haase & Amthor, 2020). Besides, these two models need much longer training steps; for example, the model proposed by Krivan et al. (2020) took 1200 epochs with a batch size of 32 per GPU to achieve state-of-the-art accuracy. The model proposed in this article intends to address these drawbacks in more a manageable way.

Inspired by the transformer architecture and the Deep Speech model, the current study employs the concepts of customized ResNets and Bi-GRUs with layer normalization to reduce the computational costs further while maintaining a very interesting performance without any need for external language models.

### 3. Methodology

This section presents the formulation of the proposed automatic speech recognition model architecture based on customized ResNets with Bi-GRUs.

#### 3.1. Mel spectrogram

Humans cannot recognize frequencies on a linear scale. Lower frequency differences are easier to notice than higher frequency variances; for example, humans can readily distinguish between 1000 and 1500 Hz, but would struggle to distinguish between 10,000 and 10,500 Hz, although the difference is the same (Qasim, Fried, & Jacobs, 2021). The Mel scale was proposed in 1937 to solve this problem. It demonstrates that a logarithmic transformation (natural log or 10 base log) can serve the purpose of making sounds of equal distance to be of equal distance to humans (Thys, Treviño, & Nadkarni, 2021). Accordingly, the transformation from the Hertz scale to the Mel Scale is defined as:

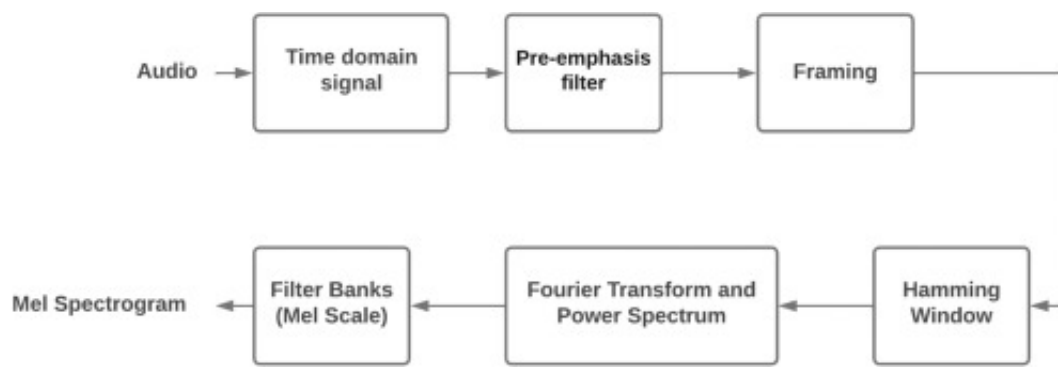
$$M = 2595 \log (1 + f/700) . \quad (1)$$

In the Mel spectrogram, the frequency components of the audio signal are converted to the Mel scale. First, the raw audio data are represented as a time-domain signal. Then, a pre-emphasis filter is used to bring the frequency spectrum into equilibrium; generally, the higher frequencies possess smaller magnitudes than the lower frequencies and make the Fourier transform operation numerically efficient. After pre-emphasis, the signal is split into multiple short-time frames, followed by the application of a window function, i.e., the Hamming window, to each frame with the following form:

$$W_f [x] = 0.54 - 0.46 \cos (2\pi x/N - 1) , \quad (2)$$

where  $0 \leq x \leq (N - 1)$  and  $N$  is the window length.





[Download: Download high-res image \(104KB\)](#)

[Download: Download full-size image](#)

Fig. 2. Process of converting audio data to the Mel Spectrogram.

The frequency and power spectrum are then obtained using an n-point Fast Fourier Transform (FFT) on each frame. The final step is to apply the triangular filters to the power spectrum on a Mel-scale to extract frequency bands (Hwang et al., 2020), as can be perceived from the block diagram shown in Fig. 2.

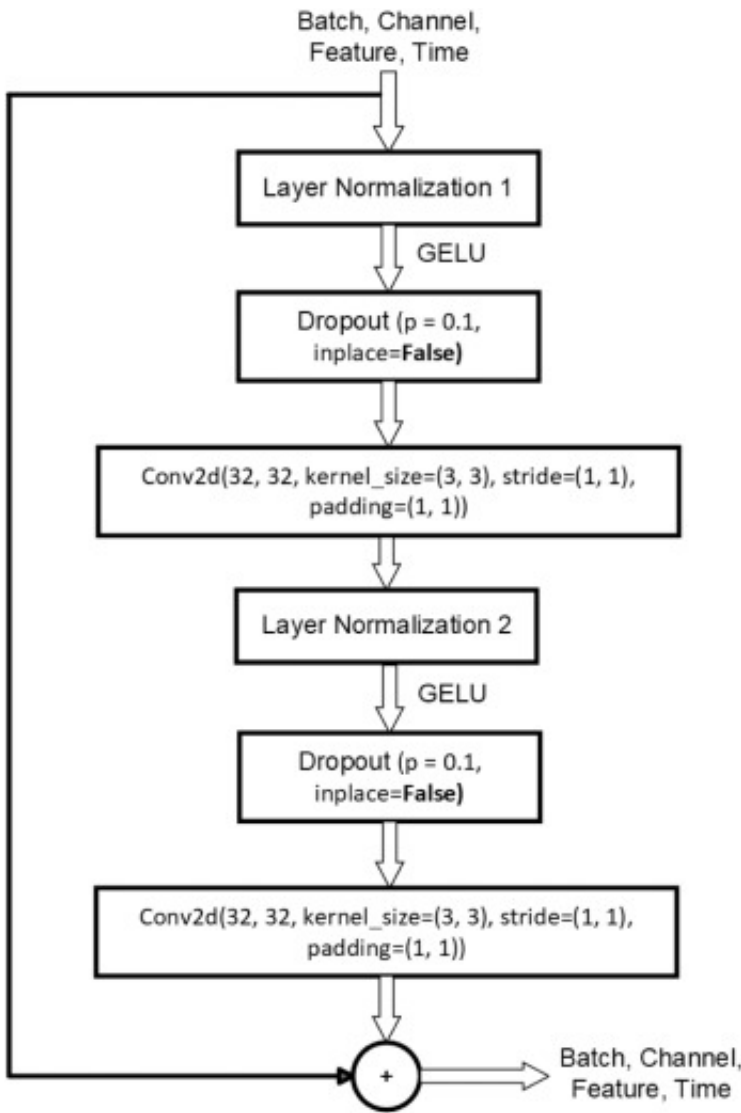
### 3.2. Residual neural network

ResNets employs skip connections to leap over some layers, usually implemented by skipping double or triple layers with non-linearities, e.g., ReLU, and batch normalization between the skipped layers. This technique avoids vanishing gradients and accuracy saturation problems, which occur when more layers are added to a sufficiently deep model that ends up with an increased training error (Szegedy, Ioffe, Vanhoucke, & Alemi, 2017).

Let  $z$  be the input. Then,  $z$  will travel through different layers, i.e., Convolution, Dropout and normalization layers, of each block of the ResNets cell. To get the output,  $y$ , the loss function is calculated on the basis of input,  $z$ , using  $y = f(z)$ , where  $f(z)$  is the loss function. The input,  $z$ , is added to the output using  $y = f(z) + z$  following the skip connection. The goal is to make  $f(z) = 0$  so that the network can learn from the difference between the input and the output.

After several trial-error tests, within each ResNets block (identity blocks), the convolution kernel of  $3 \times 3$  shape and 32 filters provided the best overall performance. It also included two Dropout layers, with each having a 10% probability ( $p = 0.1$ ) of randomly making some components of the input tensor equal to zero, and two-layer normalization within each ResNets block. The used ResNets architecture is illustrated in Fig. 3.





Download: [Download high-res image \(234KB\)](#)

Download: [Download full-size image](#)

Fig. 3. ResNets architecture of the proposed model based on the concept of skip connections.

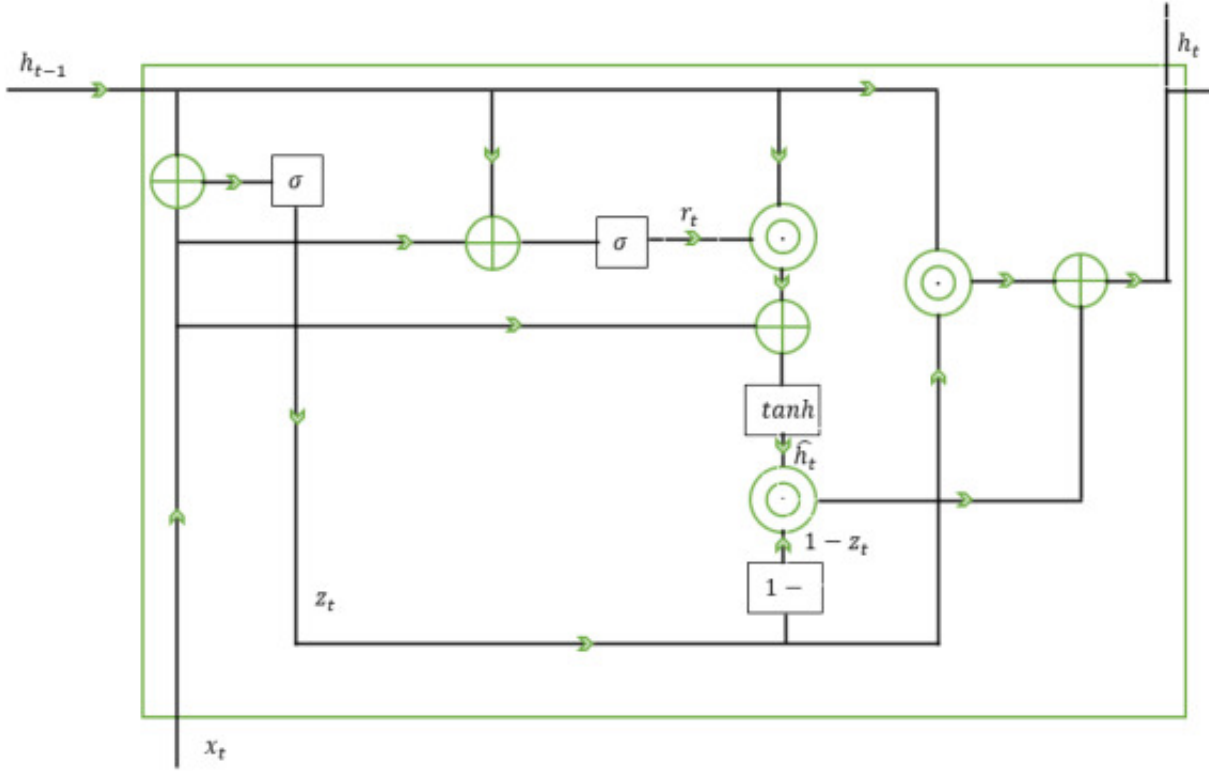
### 3.3. Bi-directional Gated Recurrent Units

Gated recurrent units (GRUs) are an updated version of the long short-term memory (LSTM) network that recognize sequential or temporal information uniquely and more straightforwardly (Chung, Gulcehre, Cho, & Bengio, 2014). GRUs combine two gates, the update and the reset gates, with a different working mechanism than LSTM. For instance, the GRU's update gate combines the LSTM's forget and input gates, but the reset gate is the same. Like the LSTM network, a GRU model modifies the information inside the units, but it does not require different memory cells (see Fig. 4).

Let  $x_t$ ,  $z_t$ ,  $r_t$ ,  $\hat{h}_t$ ,  $h_{t-1}$ , and  $h_t$  be the current input, update gate, reset gate, candidate

hidden state of the currently hidden node, hidden state at the previous moment, and current hidden state, respectively. The update gate,  $z_t$ , determines how much information in  $h_{t-1}$  will be passed along the future, and is formulated as:

$$z_t = \sigma(W_{zx}x_t + U_{zh}h_{t-1}). \quad (3)$$



[Download: Download high-res image \(204KB\)](#)

[Download: Download full-size image](#)

Fig. 4. A simplified architecture of a GRU cell ( $\sigma$ ,  $\oplus$  and  $\odot$  represent the *sigmoid* activation function, summation and Hadamard product, respectively).

Here, both  $x_t$  and  $h_{t-1}$  are multiplied by their  $W_{zx}$  and  $U_{zh}$  weights, respectively, which are learned during the algorithm training. Both results are added together and passed through a *sigmoid* function to determine their values between 0 and 1. A value of 0 means that the information will be forgotten, otherwise is retained in the current hidden state. The reset gate,  $r_t$ , decides how much of the past information to forget, and is formulated as:

$$r_t = \sigma(W_{rx}x_t + U_{rh}h_{t-1}). \quad (4)$$

Eq. (4) is identical to Eq. (3) for the update gate with the exception of the weights and the use of gates. The candidate hidden state of the currently hidden node,  $\hat{h}_t$ , uses the reset gate to store relevant information from the past using:

$$\hat{h}_t = \tanh(W_{hx}x_t + r_t \odot U_{hh}h_{t-1}). \quad (5)$$

Here, firstly,  $x_t$  and  $h_{t-1}$  are multiplied by their  $W_{hx}$  and  $U_{hh}$  weights and secondly, the Hadamard product ( $\odot$ ), i.e., the element-wise multiplication, between  $r_t$  and  $U_{hh}h_{t-1}$  is computed. The results of these two steps are summed up and passed through a non-linear activation function, for example,  $\tanh$  function, to get  $\hat{h}_t$ . The final step involves the calculation of the current hidden state,  $h_t$ , using:

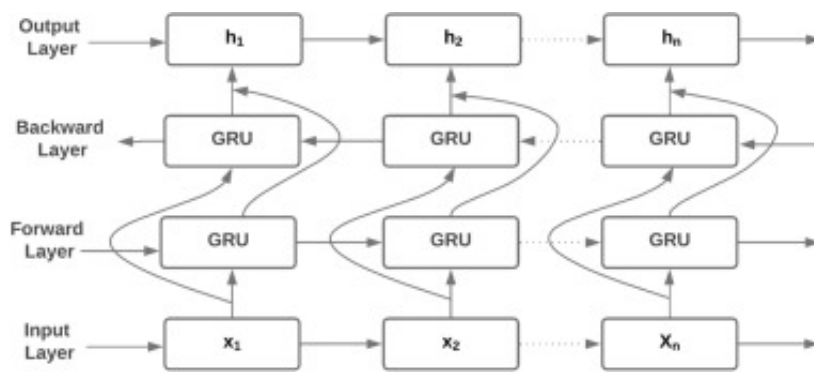
$$h_t = (1 - z_t) \odot \hat{h}_t + z_t \odot h_{t-1}. \quad (6)$$

Eq. (6) calculates the Hadamard products (element-wise multiplications) between  $(1 - z_t)$  and  $\hat{h}_t$ , and  $z_t$  and  $h_{t-1}$ , respectively. The inability of unidirectional GRUs to simultaneously consider past and future data is detrimental to the improvement of accuracy (Li et al., 2020). Hence, this research adopted a Bi-GRU structure to meet the objectives. Fig. 5 presents a Bi-GRU schematic diagram.

The input data sequence is first concurrently fed into a forward GRU,  $\vec{GRU}$  and a backward GRU,  $\overleftarrow{GRU}$ . Then, the concatenation of the hidden states  $\vec{h}_t$  and  $\overleftarrow{h}_t$ , which are produced by the forward and the backward GRU at time  $t$ , respectively, will yield the hidden state at that instant using:

$$h_t = \text{concatenate} \left( \vec{h}_t, \overleftarrow{h}_t \right), \quad (7)$$

where  $\vec{h}_t = \vec{GRU}(h_{t-1}, x_t)$  and  $\overleftarrow{h}_t = \overleftarrow{GRU}(h_{t+1}, x_t)$ .



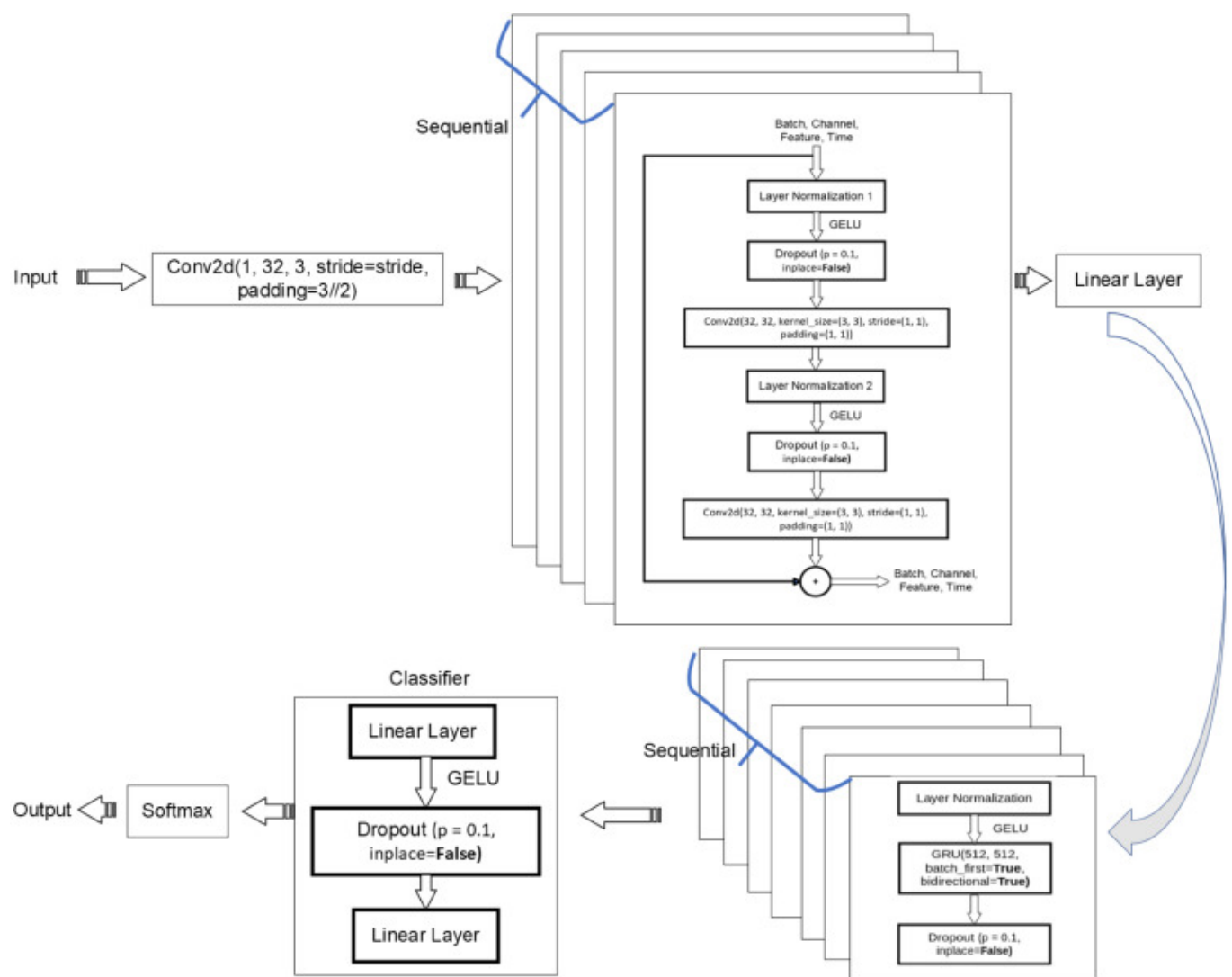
[Download: Download high-res image \(114KB\)](#)

[Download: Download full-size image](#)

Fig. 5. Schematic diagram of a Bi-GRU architecture.

### 3.4. Speech recognition model

The first step of the proposed speech recognition model is to transform the raw audio into Mel Spectrograms. Then, the model maps the character labels for each audio sample into integer labels. The heart of the model consists of two deep neural network modules:  $N = 5$  layers of ResNets architecture according to the description of Section 3.2, and a stack of 7 Bi-GRUs blocks using both a layer normalization and a dropout layer with  $p = 0.1$  within each block. The ResNets extracted and learned relevant audio features, and the Bi-GRUs block used those learnt audio features for the speech recognition task. A sequential model containing two linear layers, a GELU layer and a dropout layer, provided the best results after a few trial-error tests for classifying characters per time step. Fig. 6 illustrates the architecture of the proposed model. The logarithmic *softmax* was integrated into the output layer to get the probability matrix of characters. The final step to predict the most likely character is to use a decoder. Developing a greedy decoder allowed the extraction of the most likely characters spoken from the audio for this study.



Download: [Download high-res image \(458KB\)](#)

Download: [Download full-size image](#)

Fig. 6. Architecture of the proposed model for speech recognition.

---

## 4. Experiments

The training of the proposed model was accomplished using an NVIDIA DGX Station with 4 NVIDIA Tesla V100 tensor core GPUs having 128GB of memory, the CUDA library in its 10.1 version, and the open-source PyTorch machine learning library for Python coding. The code implementation comprised five steps. First, the datasets were collected using the 'torchaudio.datasets' module. The PyTorch 'DataLoader' function facilitated the building of the train and test datasets. The second step included mapping the characters to integers and vice-versa by defining a 'Text\_Transformation' class. It also performed data processing tasks. The 'torchaudio.transforms' module transformed the data samples' waveform into Mel Spectrograms and a pre-defined function using 'torch.nn.utils.rnn.pad\_sequence' helped to pad the Spectrograms and labels (from the transcriptions) into equal-length sequences. This step was concluded by defining a greedy decoder that returned the decoded and target transcriptions. The third step comprised defining the model's architecture. This study built a class named 'Layer\_Norm' using the 'torch.nn' module for the CNNs input. Then, a customized ResNets class was built using the 'torch.nn' module. A pre-defined forward function within the class returned the batch, channel, feature, and time utilizing Layer\_Norm, GELU, 2-dimensional CNN and dropout layers based on residual connection. For determining the Bi-GRU cells, this study built a class named 'Bi\_Directional\_GRU' using the 'torch.nn' module. Within it, a pre-defined forward function returned the Bi-GRU cell based on Layer\_Norm, GELU, Bi-GRU and dropout layers. The last part of the third phase combined all the previously defined classes into a speech recognition class, including a classifier comprised of Linear, GELU and dropout layers using the 'torch.nn.Sequential' module. A pre-defined forward function returned the batch, time, and features for classification tasks. The fourth stage covered the model training procedures. A pre-defined function named 'model\_train' was used for model training purposes, and the Spectrograms and labels were fed into the model. Initially, the 'torch.optim' module aimed to set the gradients to zero. The logarithmic *softmax* from the 'torch.nn.functional' module worked as the output layer to determine the probabilities of each character of the output transcriptions. The CTC loss function from the 'torch.nn' module calculated the probability of the correct labels. The fifth phase included the testing procedures of the model. The python 'jiwer' package facilitated calculating the WER and CER metrics. The previously defined greedy decoder picked up the character with the highest probability from the output of the logarithmic

*softmax* layer using 'torch.nn.functional' class.

## 4.1. Connectionist temporal classification loss

The CTC Loss function calculates a differentiable loss value for each input node by adding all potential input alignments to the target. It employs a bidirectional LSTM design with peepholes and forgets gates having logistic *sigmoid* ranging from 0 (zero) to 1 (one). The hyperbolic tangent function is necessary for activation. The main difference between the CTC and other temporal classifiers is that CTC segments the input sequences indirectly, which has several advantages, including avoiding the need to locate label borders that are inherently ambiguous and allowing grouping of the label predictions (Zeyer, Irie, Schlüter, & Ney, 2018).

If  $I$  and  $O$  are the input and output, respectively, then the CTC objective for  $(I, O)$  is defined as:

$$p(O|I) = \sum_{X \in X_{I,O}} \prod_{t=1}^T p_t(x_t|I), \quad (8)$$

where  $p(O|I)$  and  $p_t(x_t|I)$  represent CTC conditional probability and per time-step probability for a single alignment, respectively (Graves, Fernández, Gomez, & Schmidhuber, 2006).

## 4.2. Dataset

Due to good baselines for character-based ASR, the LibriSpeech corpus (Panayotov, Chen, Povey, & Khudanpur, 2015) was used to assess the proposed model. This study utilizes all the three training datasets: *Train\_Clean\_100*, *Train\_Clean\_360*, and *Train\_Other\_500* for the model training, and the test dataset was *Test\_Clean*. The number suffix in the dataset name denotes the dataset's time in hours.

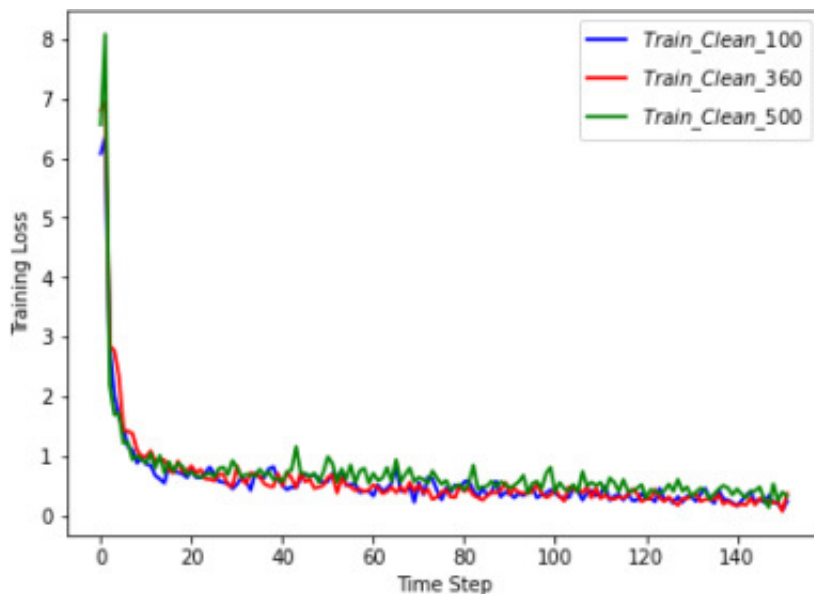
Also, to investigate the natural generalization of the proposed model, i.e., to verify that it can perform well on other datasets, the current study used the LJ Speech dataset (Ito & Johnson, 2017), which contains around **13K** short audio clips from a single speaker spanning over approximately **24** hours.

## 4.3. Model training

The following hyper-parameters provided the best results after several trial-error tests: *batch\_size* = 10,  $p$  = 0.1, *epochs* = 100,  $\alpha$  =  $5.0E - 4$ ,  $N$  = 29/31, *N\_cnn\_layers* = 5, *F\_N* = 128, *N\_rnn\_layers* = 7, *D\_rnn* = 512, and

$stride = 2$ . Here,  $p$ ,  $\alpha$ ,  $N$ ,  $N_{cnn\_layers}$ ,  $F_N$ ,  $N_{rnn\_layers}$  and  $D_{rnn}$  denote the likelihood of a component being zeroed within the dropout layers, the learning rate, number of classes (**29** and **31** for the LibriSpeech and LJ Speech datasets, respectively), number of ResNets layers, number of features, number of Bi-GRU layers, and GRU dimension, respectively.

Spectrograms and labels were fed into the model to train it. AdamW was used as the optimizer because traditional Adam contains a wrong implementation of weight decay [Loshchilov & Hutter, 2017](#). Furthermore, the CTC loss function was applied to calculate the loss. For the LibriSpeech corpus, it was trained on three different train datasets: *Train\_Clean\_100*, *Train\_Clean\_360*, and *Train\_Clean\_500*. Each contains 100, 360 and 500 hours long audio clips. On the other hand, for the LJ Speech dataset, the number of audio samples of the train and test datasets were 11,790 and 1310, respectively. The algorithm was trained up to 100 epochs, and training time was proportional to the volume of datasets. For example, *Train\_Clean\_500* dataset took around 1.5 h on average for each epoch. [Fig. 7](#) illustrates the training loss comparison of the proposed model for the different used datasets. Clearly, the model gradually learned without undergoing over-fitting or under-fitting problems. The minimum-maximum values of the training loss for *Train\_Clean\_100*, *Train\_Clean\_360*, and *Train\_Clean\_500* datasets were equal to (0.078 – 6.945), (0.097 – 6.351), and (0.141 – 8.078), respectively.



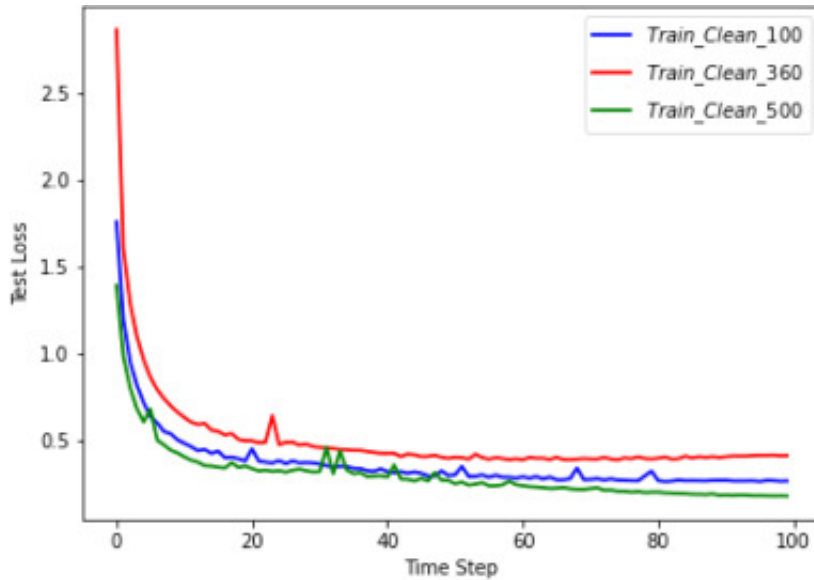
[Download: Download high-res image \(164KB\)](#)

[Download: Download full-size image](#)

Fig. 7. Obtained Training loss along time for each used dataset.



In the case of the LibriSpeech dataset, the model testing used a separate dataset from the same corpus: *Test\_Clean*. The Greedy Decoder, which selects the word/character with the highest probability, i.e., acts greedily, was implemented for decoding purposes. Fig. 8 illustrates the test loss of the model in terms of the different used datasets. The Minimum-Maximum test loss values were equal to 0.39 – 2.862, 0.266 – 1.758, and 0.182 – 1.393, for *Train\_Clean\_100*, *Train\_Clean\_360*, and *Train\_Clean\_500* datasets, respectively.



[Download: Download high-res image \(152KB\)](#)

[Download: Download full-size image](#)

Fig. 8. Obtained test loss along time for each used dataset.

## 5. Results

The model evaluation used state-of-the-art metrics for speech recognition tasks: Word Error Rate and Character Error Rate. It is to be noted that, in the literature, almost all the researchers use these two metrics for ASR model evaluation.

### 5.1. Word and character error rates

These two error functions measure how much a model misreads text in a transcription. WER computes the Levenshtein distance (Yujian & Bo, 2007) between the reference sequence and hypothesis sequence in word level, while CER computes it in character level. If  $S_w$  is the number of words substituted,  $D_w$  the number of words deleted,  $I_w$  the number of words inserted, and  $N_w$  the number of words in the reference, then WER is:

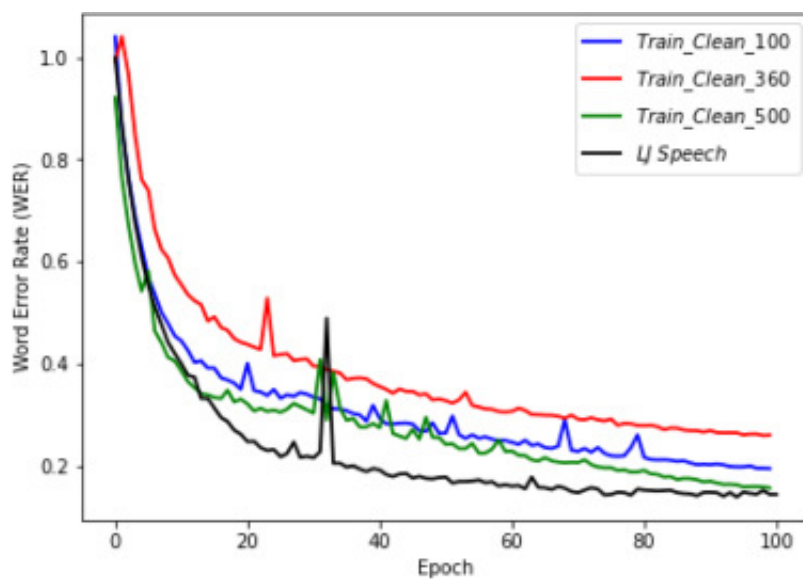
$$WER = (S_w + D_w + I_w) / N_w. \quad (9)$$

Similarly, if  $S_c$  is the number of characters substituted,  $D_c$  the number of characters deleted,  $I_c$  the number of the character inserted, and  $N_w$  the number of characters in the reference, then CER can be expressed as:

$$CER = (S_c + D_c + I_c) / N_c. \quad (10)$$

The lower these error metric values, the better the model's performance, with a CER or WER of 0 (zero) being a perfect score. Because words, unlike characters, are of varying lengths and do not offer a clear comparison, the WER value might have little importance for the model's quality, i.e., a word is already incorrectly recognized if just one letter in it is not correct. As a result, the WER should rarely describe a model's worthiness.

This study performed the model evaluation based on the aforementioned experimental setups and parameters. [Fig. 9](#), [Fig. 10](#) present the WER and CER values obtained by the proposed model for the different used datasets. After a few trial-error tests, the obtained hyper-parameters described in [Section 4.3](#) provided the following model outcomes. For the LibriSpeech dataset, the minimum value of WER of the model were equal to 25.9, 19.4, and 15.6%, and for CER, they were 8.3, 6.0, and 4.7% on *Train\_Clean\_100*, *Train\_Clean\_360*, and *Train\_Clean\_500* datasets, respectively. The tested dataset was the *Test\_Clean*, which contains 5.4 h of clean audio speech in all those cases. On the other hand, based on the test dataset obtained from data splitting as described in [Section 4.3](#), the model exhibited minimums CER and WER of 3.61 and 13.86% on the LJ Speech dataset, respectively, as can be verified in [Table 1](#). The model was trained up to 100 epochs for each of the aforementioned cases, and it might even exhibit better performance gains for training over 100 epochs.

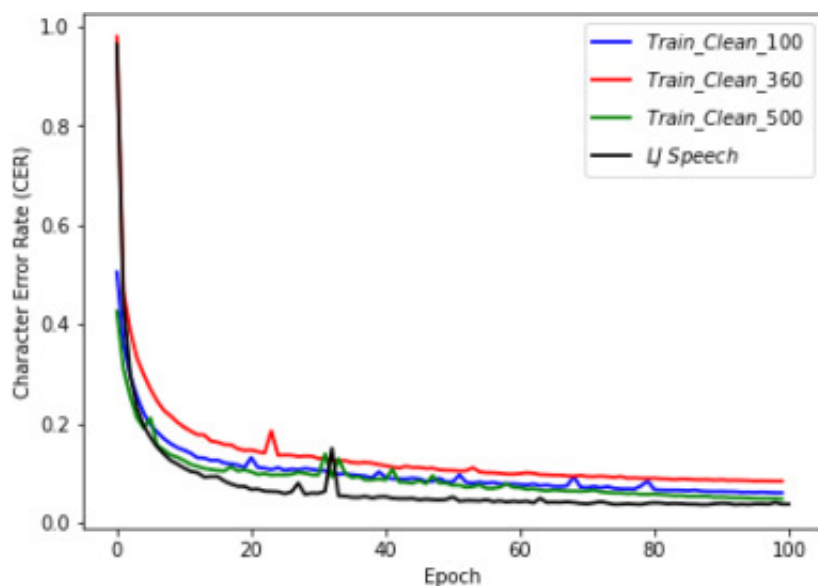


[Download: Download high-res image \(193KB\)](#)

[Download: Download full-size image](#)

Fig. 9. Obtained word error rate for each used dataset.

The proposed model demonstrated its effectiveness in predicting the target transcriptions accurately. Table 2 depicts the model's performance in predicting the speech transcriptions on the LJ Speech dataset. Two arbitrary target and predicted transcriptions were taken from the test dataset when the CER and WER values were 3.7 and 14.39%, respectively. The second row presents the correctly predicted transcriptions, and the last contains three wrongly predicted words within the target. The model failed to correctly predict characters within the words 'our', 'cocked our', and 'swords'. Nevertheless, the proposed model acquired a minimum CER of only 3.61% on the LJ Speech dataset, meaning that 96.39% characters, including letters, punctuation, special symbols and spaces of the target transcriptions, were correctly predicted. Hence, the results of Table 2 justify its effectiveness in the current study context.



[Download: Download high-res image \(162KB\)](#)

[Download: Download full-size image](#)

Fig. 10. Obtained Character error rate for each used dataset.

Table 1. Performance comparison in terms of different metrics on different datasets.

| Train datasets         | CER (MIN%) | WER (MIN%) | Duration         |
|------------------------|------------|------------|------------------|
| <i>Train_Clean_100</i> | 8.3        | 25.9       | 40 : 26<br>: 24  |
| <i>Train_Clean_360</i> | 6.0        | 19.4       | 71 : 35<br>: 22  |
| <i>Train_Clean_500</i> | 4.7        | 15.6       | 158 : 51<br>: 15 |
| <b>LJ Speech</b>       | 3.61       | 13.86      | 21 : 15<br>: 27  |

Number of model parameters = **33196445**, duration is represented in hour : minute : second).

Table 2. Prediction performance of the model on the LJ Speech dataset.

| Attributes         | Transcription                                                                                                                                                                       |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Target 1</b>    | <i>that he compared the four empty cartridge cases found near the scene of the shooting with a test cartridge fired from the weapon in Oswald's possession when he was arrested</i> |
| <b>Predicted 1</b> | <i>that he compared the four empty cartridge cases found near the scene of the shooting with a test cartridge fired from the weapon in Oswald's possession when he was arrested</i> |

**Target 2**    *in an instant our faces were covered we cocked our pistols and with drawn swords stood waiting to receive the enemy*

---

**Predicted**    *in an instant **are** faces were covered we **cockedar** pistols and with drawn **sords** stood*  
**2**                *waiting to receive the enemy*

---

The bold words represents wrongly predicted words.

---

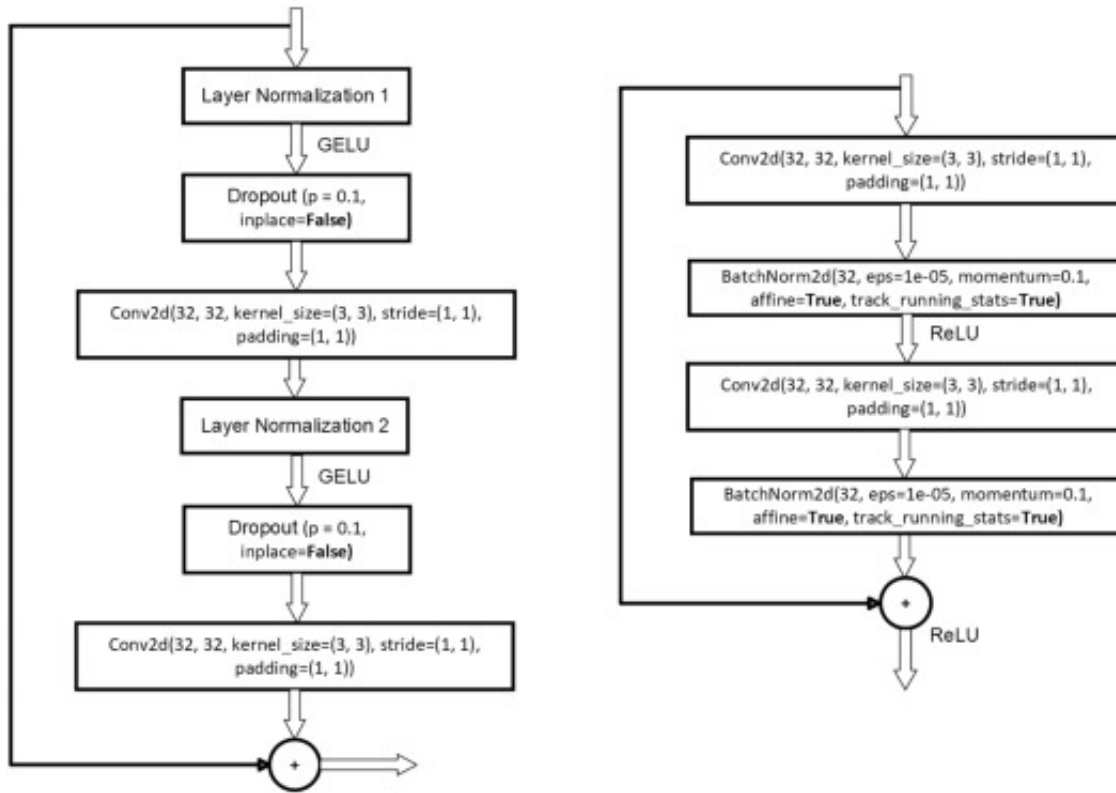
## 6. Discussion

This study aimed to build a deep neural network-based model that is computationally efficient and produces good outcomes. As a solution, this research suggests five layers of customized ResNets with seven layers of a Bi-GRUs-based ASR model. The LibriSpeech ASR corpus, which has 1000 hours of audio speech, and the LJ Speech datasets, were used for training and testing the proposed model.

During the development of the proposed model, different architectures of the ResNets layers were examined. The layer normalization instead of bath normalization within the ResNets block produced better results. The batch normalization in the network reduced both the train and test losses; nonetheless, there was a significant reduction in terms of WER and CER. Applying ResNets without batch normalization layers improved the CER and WER by 73.3 and 53.93%, respectively, reducing the train and test losses by 12.46 and 36.95%, respectively. Fig. 11 presents two different examined architectures of the ResNets layers. The left residual block consists of two-layer normalization, dropout, convolution layers using the GELU activation function, and a skip connection according to the provided arrangement. Furthermore, the residual block on the right employs two 2-dimensional (2D) batch normalization convolution layers using the ReLU activation function and a skip connection according to the illustrated arrangement. Interestingly, the residual block on the left provided much better results; hence, the proposed model implements the ResNets architecture according to it.

This study suggests replacing batch normalization layer with layer normalization to achieve better ASR accuracy. As the batch size highly influences batch normalization, the proposed model was trained with different batch sizes to examine whether the obtained good performances are sustained with an increase in the used batch size. A batch normalization layer must compute the mean and variance to normalize the previous outputs across the batch. Hence, for small batch sizes, its estimation will be

less accurate. Here, layer normalization is proposed to address the shortcoming. This study examined three different batch sizes: **10**, **16** and **32**. The variations of different batch sizes exhibited some small effects on the model performance, as can be seen in Fig. 12. For *Train\_Clean\_100* and *Test\_Clean* as the train and test datasets, the proposed model achieved a slight improvement in terms of CER by 4.4 and 7.56% with a batch size of **10** compared to its value of **16** and **32**, respectively. Overall, the model is capable of maintaining its good performances regardless of the batch size.

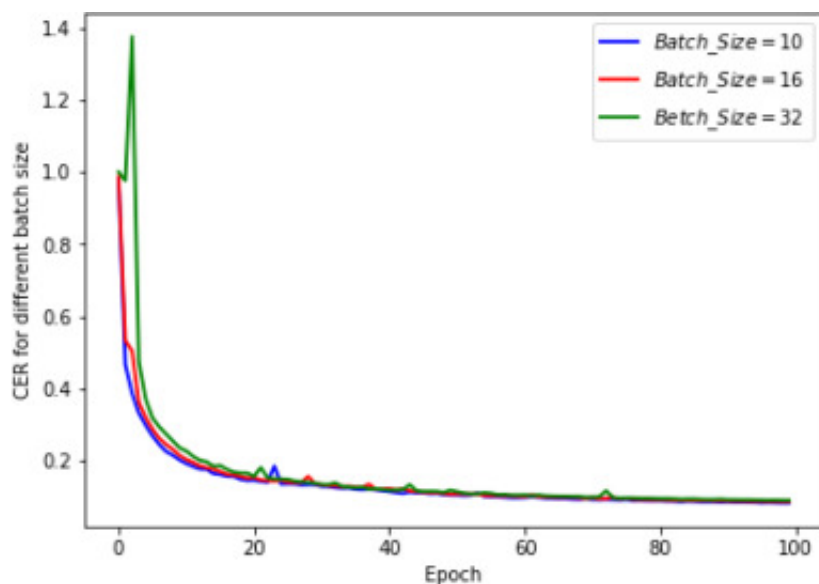


[Download: Download high-res image \(276KB\)](#)

[Download: Download full-size image](#)

Fig. 11. Model comparison as to the used residual neural network architecture.

The training data significantly influences the model's accuracy and increases the computational demands. The training on the *Train\_Clean\_500* dataset containing **500** hours of English audio speech led to the best performance. For **500** hours of audio data, the CER had an increase of 43.37 and 26.67% relatively to the **100** and **500** hours of audio data, respectively.



[Download: Download high-res image \(130KB\)](#)

[Download: Download full-size image](#)

Fig. 12. Obtained CER for different values of batch size.

The number of ResNets and Bi-GRUs layers also demonstrated performance variation. Using **3** and **5** layers of ResNets and Bi-GRUs resulted in a 79.12% performance degradation in terms of CER compared to the current **5** and **7** layers of the parameters previously mentioned. However, in [Wang et al. \(2018\)](#), it was demonstrated that the increase of the GRUs layers beyond **5** or more fails to enhance the performance of a model based on Deep Speech 2 ([Amodei et al., 2016](#)). The opposite findings of the current study are due to the insertion of the proposed ResNets layers.

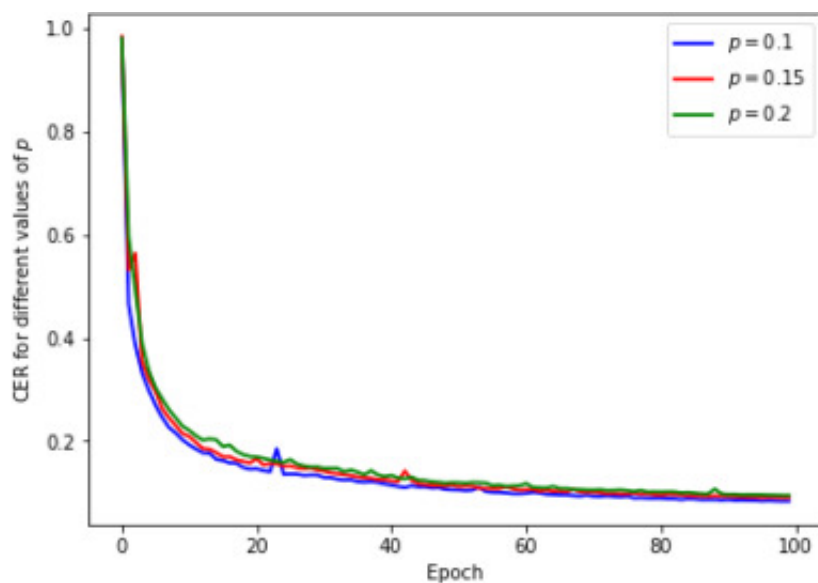
Replacing Adam for AdamW as the optimizer, and GELU instead of ReLU as the activation function, demonstrated their usefulness in the problem under study. AdamW overcame the limitation of Adam of its wrong implementation of the weight decay, and hence, it showed better performance. GELU combines approximated numbers and various functions, e.g., hyperbolic tangent (*tanh*), and has a negative coefficient that changes to a positive coefficient as time goes on; for example, when  $x$  is more significant than zero, the output will be  $x$ , except for the range **0** (zero) to **1** (one), which tends toward a smaller  $y$  value. Because of its formulation, GELU showed better results than ReLU.

This study also examined the model performance in terms of different values of  $p$ , which is defined as a measure of the likelihood of a component being zeroed within the dropout layers, as can be seen in [Fig. 13](#). The model performance seems to decrease gradually with the increase in  $p$  values. In terms of statistics, for



*Train\_Clean\_100* and *Test\_Clean* as the train and test datasets, the model achieved 7.98 and 12.2% of improvements in terms of CER with a  $p$  of 0.1 compared to its value of 0.15 and 0.20, respectively.

The current study also trained the proposed model on the LibriTTS dataset (Zen et al., 2019), and the performance was less impressive compared to the LibriSpeech and LJ Speech datasets, which can be seen as a drawback of the proposed model. The LibriTTS contains 49 different characters instead of 29 and 31 of the used datasets. Text mapping is one of the crucial steps for effective speech recognition. With the increase in the number of characters in datasets, the proposed model tends to become less effective in terms of accuracy. For example, the special characters ", [ , ; , ? , } and / were difficult to correctly predict. So, the exhibited poor performance of the model, in this case, was due to the increased number of characters in the data samples.



[Download: Download high-res image \(125KB\)](#)

[Download: Download full-size image](#)

Fig. 13. Obtained CER for different values of  $p$ .

Compared with some already published state-of-the-art models, the proposed model requires fewer parameters while maintaining excellent CER values on the LibriSpeech and LJ Speech datasets. However, the application of depthwise separable convolution layers could also reduce the computational costs by a more significant margin (Han et al., 2020 and Krizan et al., 2020). The authors argue that this approach is likely to fail to learn properly during training if the network size is small. Also, a tremendous amount of training steps (around 1200 epochs) are required to acquire state-of-the-art accuracy. The currently proposed model achieved good accuracy with only 100 epochs of training and is suitable for any network regardless of its size. Table 3 depicts a

comprehensive comparison of the proposed model with some already published research based on the LibriSpeech corpus. The comparisons were made regarding the number of parameters, CER and WER values. It exhibited the best CER value among the mentioned works. However, the obtained WER values are less competitive than other baselines under consideration, which is the future scope of this research.

Table 3. Performance comparison of different published models on LibriSpeech *Test\_Clean* dataset.

| Models                                     | Parameters (M) | WER (%) | CER (%)     |
|--------------------------------------------|----------------|---------|-------------|
| <a href="#">Likhomanenko et al. (2021)</a> | 270            | 2.8*    | N/A         |
| <a href="#">Park et al. (2019)</a>         | 360            | 2.8*    | N/A         |
| <a href="#">Synnaeve et al. (2019)</a>     | 270            | 2.89    | N/A         |
| <a href="#">Karita et al. (2019)</a>       | 130            | 18.0    | 8.4         |
| <a href="#">Moritz et al. (2019)</a>       | 70             | 5.4     | 6.6         |
| <a href="#">Wang et al. (2018)</a>         | 35             | 24.55   | 7.06        |
| <b>Proposed model</b>                      | <b>33*</b>     | 15.6    | <b>4.7*</b> |

The values with\* represent the best performance and N/A means not available, i.e., the authors did not use CER for model evaluation.

## 7. Conclusion

This study aimed to develop an effective ASR model without the requirement of any external language model to provide good results while relying on fewer computational resources. ResNets’s suitability for smoothing model training and giving excellent performances was validated. In addition, layer normalization instead of batch normalization was shown to be more effective in ASR tasks in improving the model’s performance. For optimizer, AdamW, instead of traditional Adam, demonstrated better results. Also, the number of ResNets and Bi-GRU layers used in the model played a crucial role in acquiring competitive performance. The number of training datasets greatly influenced the achievement of the best performance. For example, the model achieved an improvement of CER by 43.37 and 26.67% while using 500 hours of audio data relative to the other used datasets. The proposed model demonstrated superior performance using less computational costs in CER by surpassing previous state-of-

the-art models on the LibriSpeech datasets. However, the proposed model was less effective than other baselines regarding WER. A word is already incorrectly recognized if just one letter in it is not correct. Hence, with the tremendous results on CER (4.7% means 95.3% characters, i.e., letters, punctuation and spaces, were correctly recognized), it should be enough to justify the efficiency of the model. Henceforth, the future emphasis would be given to employing the model effectively on other datasets, for example, on the MLS dataset suggested in [Pratap, Xu, Sriram, Synnaeve, and Collobert \(2020\)](#), and the introduction of a 1D CNN network to further reduce the computational costs.

## CRedit authorship contribution statement

**Selim Reza:** Investigation, Data collection, Code implementation, Formal analysis, Writing – original draft. **Marta Campos Ferreira:** Writing – review & editing. **J.J.M. Machado:** Writing – review & editing. **João Manuel R.S. Tavares:** Conceptualization, Funding acquisition, Supervision, Writing – review & editing.

## Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgements

This article results from the project Safe Cities– “Inovação para Construir Cidades Seguras”, with reference [POCI-01-0247-FEDER-041435](#), co-funded by the European Regional Development Fund (ERDF), through the Operational Programme for Competitiveness and Internationalization (COMPETE 2020) , under the PORTUGAL 2020 Partnership Agreement.

[Recommended articles](#)

## Data availability

The data is public available.

## References

[Amodei et al., 2016](#) Amodei D., Ananthanarayanan S., Anubhai R., Bai J., Battenberg E., Case C., et al.

## Deep speech 2: End-to-end speech recognition in english and mandarin

International conference on machine learning, PMLR (2016), pp. 173-182

[Google Scholar ↗](#)

[Benzeghiba et al., 2007](#) Benzeghiba M., De Mori R., Deroo O., Dupont S., Erbes T., Jouvét D., et al.

## Automatic speech recognition and speech variability: A review

Speech Communication, 49 (10–11) (2007), pp. 763-786



[View PDF](#)

[View article](#)

[View in Scopus ↗](#)

[Google Scholar ↗](#)

[Chan et al., 2016](#) Chan W., Jaitly N., Le Q., Vinyals O.

## Listen, attend and spell: A neural network for large vocabulary conversational speech recognition

2016 IEEE international conference on acoustics, speech and signal processing, IEEE (2016), pp. 4960-4964

[View in Scopus ↗](#) [Google Scholar ↗](#)

[Chung et al., 2014](#) Chung J., Gulcehre C., Cho K., Bengio Y.

## Empirical evaluation of gated recurrent neural networks on sequence modeling

(2014)

arXiv preprint [arXiv:1412.3555 ↗](#)

[Google Scholar ↗](#)

[Fan et al., 2021](#) Fan R., Chu W., Chang P., Xiao J., Alwan A.

## An improved single step non-autoregressive transformer for automatic speech recognition

Proceedings of the annual conference of the international speech communication association, INTERSPEECH, vol. 2 (2021), pp. 1406-1410, [10.21437/Interspeech.2021-1955](#)

↗

[View in Scopus ↗](#) [Google Scholar ↗](#)

[Graves et al., 2006](#) Graves, A., Fernández, S., Gomez, F., & Schmidhuber, J. (2006).

Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd international conference on machine learning* (pp. 369–376).

[Google Scholar ↗](#)

[Haase and Amthor, 2020](#) Haase, D., & Amthor, M. (2020). Rethinking depthwise separable convolutions: How intra-kernel correlations lead to improved mobilenets. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 14600–14609).

[Google Scholar ↗](#)

[Han et al., 2020](#) Han, W., Zhang, Z., Zhang, Y., Yu, J., Chiu, C.-C., Qin, J., et al. (2020). ContextNet: Improving Convolutional Neural Networks for Automatic Speech Recognition with Global Context. In *Proc. interspeech 2020* (pp. 3610–3614).

[Google Scholar ↗](#)

[Hannun et al., 2014](#) Hannun A., Case C., Casper J., Catanzaro B., Diamos G., Elsen E., et al. Deep speech: Scaling up end-to-end speech recognition (2014)

arXiv preprint [arXiv:1412.5567 ↗](#)

[Google Scholar ↗](#)

[Hwang et al., 2020](#) Hwang Y., Cho H., Yang H., Won D.-O., Oh I., Lee S.-W. Mel-spectrogram augmentation for sequence to sequence voice conversion

(2020)

arXiv preprint [arXiv:2001.01401 ↗](#)

[Google Scholar ↗](#)

[Ito and Johnson, 2017](#) Ito K., Johnson L.

The LJ speech dataset

(2017)

[https://keithito.com/LJ-Speech-Dataset/ ↗](https://keithito.com/LJ-Speech-Dataset/)

[Google Scholar ↗](#)

[Junqua and Haton, 2012](#) Junqua J.-C., Haton J.-P.

Robustness in automatic speech recognition: Fundamentals and applications

Springer Science & Business Media (2012)

[Google Scholar ↗](#)

[Karita et al., 2019](#) Karita S., Watanabe S., Iwata T., Delcroix M., Ogawa A., Nakatani T.

Semi-supervised end-to-end speech recognition using text-to-speech

## and autoencoders

ICASSP 2019-2019 IEEE international conference on acoustics, speech and signal processing, IEEE (2019), pp. 6166-6170

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[Kriman et al., 2020](#) Kriman S., Beliaev S., Ginsburg B., Huang J., Kuchaiev O., Lavrukhin V., *et al.*

### Quartznet: Deep automatic speech recognition with 1d time-channel separable convolutions

ICASSP 2020-2020 IEEE International conference on acoustics, speech and signal processing, IEEE (2020), pp. 6124-6128

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[Lee and Watanabe, 2021](#) Lee J., Watanabe S.

### Intermediate loss regularization for ctc-based speech recognition

ICASSP 2021-2021 IEEE international conference on acoustics, speech and signal processing, IEEE (2021), pp. 6224-6228

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[Li et al., 2020](#) Li P., Luo A., Liu J., Wang Y., Zhu J., Deng Y., *et al.*

### Bidirectional gated recurrent unit neural network for Chinese address element segmentation

ISPRS International Journal of Geo-Information, 9 (11) (2020), p. 635

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[Li et al., 2021](#) Li, S., Xu, M., & Zhang, X.-L. (2021). Efficient conformer-based speech recognition with linear attention. In *2021 Asia-Pacific signal and information processing association annual summit and conference* (pp. 448–453).

[Google Scholar ↗](#)

[Likhomanenko et al., 2021](#) Likhomanenko T., Xu Q., Pratap V., Tomasello P., Kahn J., Avidov G., *et al.*

### Rethinking evaluation in ASR: Are our models robust enough?

Proc. Interspeech 2021 (2021), pp. 311-315, [10.21437/Interspeech.2021-1758 ↗](#)

[View in Scopus ↗](#) [Google Scholar ↗](#)

[Liu et al., 2019](#) Liu Y., Ott M., Goyal N., Du J., Joshi M., Chen D., *et al.*

### Roberta: A robustly optimized bert pretraining approach

(2019)

arXiv preprint [arXiv:1907.11692 ↗](#)

[Google Scholar ↗](#)

[Loshchilov and Hutter, 2017](#) Loshchilov I., Hutter F.

### Decoupled weight decay regularization

(2017)

arXiv preprint [arXiv:1711.05101 ↗](#)

[Google Scholar ↗](#)

[Majumdar et al., 2021](#) Majumdar S., Balam J., Hrinchuk O., Lavrukhin V., Noroozi V., Ginsburg B.

### Citrinet: Closing the gap between non-autoregressive and autoregressive end-to-end models for automatic speech recognition

(2021)

arXiv preprint [arXiv:2104.01721 ↗](#)

[Google Scholar ↗](#)

[Moritz et al., 2019](#) Moritz N., Hori T., Le Roux J.

### Triggered attention for end-to-end speech recognition

ICASSP 2019-2019 IEEE international conference on acoustics, speech and signal processing, IEEE (2019), pp. 5666-5670

[Crossref ↗](#)   [View in Scopus ↗](#)   [Google Scholar ↗](#)

[Panayotov et al., 2015](#) Panayotov V., Chen G., Povey D., Khudanpur S.

### Librispeech: an asr corpus based on public domain audio books

2015 IEEE international conference on acoustics, speech and signal processing, IEEE (2015), pp. 5206-5210

[View in Scopus ↗](#)   [Google Scholar ↗](#)

[Park et al., 2019](#) Park D.S., Chan W., Zhang Y., Chiu C.-C., Zoph B., Cubuk E.D., *et al.*

### SpecAugment: A simple data augmentation method for automatic speech recognition

Proc. interspeech 2019 (2019), pp. 2613-2617, [10.21437/Interspeech.2019-2680 ↗](#)

[View in Scopus ↗](#)   [Google Scholar ↗](#)

[Pratap et al., 2020](#) Pratap V., Xu Q., Sriram A., Synnaeve G., Collobert R.

### Mls: A large-scale multilingual dataset for speech research

(2020)

arXiv preprint [arXiv:2012.03411 ↗](#)

[Google Scholar ↗](#)

[Qasim et al., 2021](#) Qasim S.E., Fried I., Jacobs J.

## Phase precession in the human hippocampus and entorhinal cortex

Cell, 184 (12) (2021), pp. 3242-3255

[View in Scopus ↗](#) [Google Scholar ↗](#)

[Santurkar et al., 2018](#) Santurkar, S., Tsipras, D., Ilyas, A., & Madry, A. (2018). How does batch normalization help optimization?. In *Proceedings of the 32nd international conference on neural information processing systems* (pp. 2488–2498).

[Google Scholar ↗](#)

[Synnaeve et al., 2019](#) Synnaeve G., Xu Q., Kahn J., Likhomanenko T., Grave E., Pratap V., et al.

## End-to-end asr: from supervised to semi-supervised learning with modern architectures

(2019)

arXiv preprint [arXiv:1911.08460 ↗](#)

[Google Scholar ↗](#)

[Szegedy et al., 2017](#) Szegedy, C., Ioffe, S., Vanhoucke, V., & Alemi, A. A. (2017). Inception-v4, inception-resnet and the impact of residual connections on learning. In *Thirty-first AAAI conference on artificial intelligence*.

[Google Scholar ↗](#)

[Thys et al., 2021](#) Thys T.M., Treviño J., Nadkarni N.M.

## Perceptual–acoustic comparisons of natural sonic environments: Applications for nature-deprived populations

Ecopsychology, 13 (3) (2021), pp. 151-164

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[Wang et al., 2018](#) Wang K., Guan D., Li B.

## Deep group residual convolutional CTC networks for speech recognition

International conference on advanced data mining and applications, Springer (2018), pp. 318-328

[Crossref ↗](#) [View in Scopus ↗](#) [Google Scholar ↗](#)

[Yu and Deng, 2016](#) Yu D., Deng L.

## Automatic speech recognition

Springer (2016)

[Google Scholar ↗](#)



[Yujian and Bo, 2007](#) Yujian L., Bo L.

### A normalized levenshtein distance metric

IEEE Transactions on Pattern Analysis and Machine Intelligence, 29 (6) (2007), pp. 1091-1095

[View in Scopus ↗](#) [Google Scholar ↗](#)

[Zen et al., 2019](#) Zen H., Dang V., Clark R., Zhang Y., Weiss R.J., Jia Y., *et al.*

### LibriTTS: A corpus derived from LibriSpeech for text-to-speech

(2019)

arXiv preprint [arXiv:1904.02882 ↗](#)

[Google Scholar ↗](#)

[Zeyer et al., 2018](#) Zeyer A., Irie K., Schlüter R., Ney H.

### Improved training of end-to-end attention models for speech recognition

Proc. interspeech, International Speech Communication Association (2018), 10.21437/interspeech.2018-1616 [↗](#)

[Google Scholar ↗](#)

[Zhao et al., 2021](#) Zhao Y., Yang X., Wang J., Gao Y., Yan C., Zhou Y.

### BART based semantic correction for mandarin automatic speech recognition system

Proc. interspeech 2021 (2021), pp. 2017-2021, 10.21437/Interspeech.2021-739 [↗](#)

[Google Scholar ↗](#)

---

## Cited by (26)

### [Automatic Speech Recognition: A survey of deep learning techniques and approaches](#)

2025, International Journal of Cognitive Computing in Engineering

[Show abstract](#) [▼](#)

### [Classification method for folded flue-cured tobacco based on hyperspectral imaging and conventional neural networks](#)

2024, Industrial Crops and Products

*Citation Excerpt :*

...Batch normalization is a technique that normalizes input data by adjusting its mean to zero and variance to one (C. Pereira et al., 2023). The mean and variance calculated during training process are utilized for input data standardization during inference, effectively reducing overfitting risks and improving network stability (Reza et al., 2023). Moreover, this layer also addresses issues often linked to using the tanh activation function, thus speeding up model convergence (Ioffe and Szegedy, 2015)....

[Show abstract](#) ✓

## Advanced acoustic footstep-based person identification dataset and method using multimodal feature fusion

2023, Knowledge Based Systems

*Citation Excerpt :*

...Thus the audio waveform could provide complementary information for the vision-based approaches [8]. Person identification has received wide research interest from the recognition of face [9], iris [10], fingerprint [11,12], voice [13], keystroke [14], signature [15], and gait [16,17]. The study of acoustic footstep identification belongs to one sub-branch of gait recognition [18–20]....

[Show abstract](#) ✓

## Customized deep learning based Turkish automatic speech recognition system supported by language model ↗

2024, Peerj Computer Science

[Show abstract](#) ✓

## Variable STFT Layered CNN Model for Automated Dysarthria Detection and Severity Assessment Using Raw Speech ↗

2024, Circuits Systems and Signal Processing

## Intelligent GRU-RIC Position-Loop Feedforward Compensation Control Method With Application to an Ultraprecision Motion Stage ↗

2024, IEEE Transactions on Industrial Informatics



[View all citing articles on Scopus](#) ↗

